

Structural and Empirical Cryptanalysis of KOD8: A Custom Eight-Stage Symmetric Cipher

A. Fatih ULUSOY

August 2026

Abstract

KOD8 is a custom symmetric cipher that routes input through one of six input-type-specific “CipherLists,” each composing exactly eight primitive reversible transformations (encodings, XOR variants, classical substitutions, and transpositions) driven by a single eight-digit numeric key. This report presents a structural and empirical cryptanalysis of the reference implementation. We show that the cipher’s advertised eight-layer design provides far less effective key-dependent entropy than its layer count suggests: several primitives ignore the key entirely, one collapses an eight-digit (nominally 10^8) key into at most 325 distinct internal states, and the block-cipher-inspired CBC-style layer reuses the literal key string as a fixed, non-random initialization vector across every message. We further show that the cipher is deterministic (identical plaintexts always yield identical ciphertexts), unauthenticated (no integrity tag, with several primitives silently absorbing decoding errors), and that the reference application persists the selected CipherList identifier as a cleartext header, eliminating any uncertainty an attacker would otherwise face in classifying the ciphertext. A known-plaintext brute-force attack against the full nominal keyspace is implemented and benchmarked, recovering the key from a single plaintext/ciphertext pair for the plain-text CipherList in under 2.4 hours on a single CPU core using an unoptimized reference Python implementation — and in effectively constant time once the structural entropy collapses identified in this report are exploited. We conclude that KOD8 is correctly classified as a composite classical (product) cipher and a functional obfuscation layer, not as a cryptographically secure primitive, and we outline concrete remediations.

Contents

1	Introduction	2
2	System Overview	2
3	Threat Model	3
4	Per-Primitive Key-Dependence Analysis	3
5	Fixed Initialization Vector in the CBC-Style Layer	5
6	Determinism and the Absence of Semantic Security	5
7	Absence of Integrity and Authentication	6
8	Cleartext CipherList Disclosure	6
		1
9	Empirical Key Recovery	6
	9.1 Method	6
	9.2 Results	7
10	Per-CipherList Summary	7

11 Discussion	8
12 Recommendations	8
13 Conclusion	8

1 Introduction

KOD8 (“Kod” — Turkish for “code”) is a hand-designed symmetric cipher implemented in `kod8engine.py`, with a companion Tkinter front end, `kod8ui.py`, that visualizes the encryption and decryption process step by step. The engine’s own documentation frames it as an “anti-LLM and decoding agent based cipher model” — that is, a scheme designed primarily to resist casual or automated pattern-based decoding rather than to meet a formal security definition such as IND-CPA or IND-CCA2.

This framing matters for how the design should be evaluated. KOD8 is not positioned as a competitor to AES-GCM or ChaCha20-Poly1305; it is a personal cryptographic-engineering exercise. Evaluated on those terms it is a legitimate and instructive artifact: a working example of a *product cipher* in the classical sense — a chain of substitution and transposition steps, in the tradition of ciphers that predate the computational security paradigm.

The purpose of this report is to evaluate that claim rigorously. We ask three questions:

1. How much of the nominal 10^8 -keyspace entropy actually reaches each of the eight steps in a given CipherList?
2. Under a realistic threat model for how the reference implementation is used (a hardcoded module-level key, and a file format that stores the CipherList identifier in cleartext), what does an attacker actually have to solve?
3. Empirically, how long does key recovery take against the reference implementation?

All findings below are derived directly from the reference source (`kod8engine.py`, `kod8ui.py`) and validated by executing the actual engine code rather than by inspection alone.

2 System Overview

```
KOD8_KEY = "86247931"
```

KOD8 defines a single global key,

¹

an eight-character decimal string, and a fixed 29-letter Turkish alphabet used by two of its substitution primitives. Seventeen primitive operations are defined in total — two encodings (`ophex`, `opbase64`), eight XOR/substitution variants (`opxorkey`, `oprollingxor`, `opkeystreamxor`, `opvigenere`, `opatbasht`, `op_unicodeshift`, `opsbox`, `opbase36`), and seven transposition/block operations (`opraillence`, `opblockrotate`, `opsplitreverse`, `opfullreverse`, `opcolumnar`, `opblockshuffle`, `opblockxor_cbc`) — each with signature `opxxx(text, encrypt, **kwargs)` and each claimed to be self-contained and reversible.

Six *CipherLists* (CL1–CL6) each select exactly eight of these primitives, in a fixed order, keyed to an input type inferred by a heuristic auto-detector (autodetect): plain text, image, video, document, numeric/data, and an “experimental” catch-all. Encryption runs the eight steps forward; decryption runs the same eight steps in reverse with `encrypt=False`. The identifier of the *CipherList* used is returned alongside the ciphertext and is required for decryption.

The Tkinter UI (`kod8ui.py`) wraps this engine, and — critically for the threat model developed in Section

```
f.write(f"# KOD8          cipher_id: {cid}\n{enc}")
```

3 — persists encrypted output to disk in a fixed text format:

1

3 Threat Model

We evaluate KOD8 under three progressively weaker attacker assumptions, each grounded in how the reference implementation actually behaves.

T1 — Source-available (white-box).

KOD8KEY is a hardcoded module-level constant, not a user-supplied secret, a password-derived key, or a per-deployment value. Any party with access to `kod8engine.py` — including anyone who has ever received this repository, cloned it from a version-control host, or extracted it from a distributed build — possesses the key outright. Under Kerckhoffs’s principle this would normally be an acceptable assumption *provided the key itself remained secret*; here the key *is* the source file. This is the single most severe practical weakness in the system and requires no cryptanalysis at all to exploit.

T2 — Ciphertext-and-header-only, key rotated.

A more charitable model assumes a deployer has changed KOD8KEY to a private eight-digit value not present in any public source tree, and that the attacker only observes `.kod8` files or wire traffic. Because the *CipherList* identifier is stored as a cleartext header (Section 8), the attacker’s search problem is immediately reduced to: recover one eight-digit key against one *known* 8-step chain — not, as the “six independent *CipherLists*” design might suggest, a joint key-and-chain identification problem.

T3 — Known-plaintext.

The strongest assumption we exercise empirically in Section 9 is a single known plaintext/ciphertext pair, which is a routine assumption in classical cryptanalysis and realistic here: KOD8’s own self-test harness, UI placeholder text, and predictable file headers (e.g. document boilerplate, JSON key names) are all plausible cribs.

The remainder of this report shows that KOD8 does not survive T2 or T3, and trivially fails under T1.

4 Per-Primitive Key-Dependence Analysis

The chain-safety documentation in `kod8engine.py` advertises “6 *CipherLists* × 8 steps” as the source of the scheme’s strength. We traced every primitive’s use of KOD8KEY directly in source and confirmed each classification by execution. Table 1 summarizes the result: of the primitives available to the engine, six reference KOD8KEY in *no way whatsoever*, and of the remainder, several derive from only a small function of the key rather than the full eight-digit value.

Three findings from Table 1 are worth stating explicitly.

Table 1: Effective key-dependent state space per primitive, against a nominal 10^8 -key eight-digit key. “Zero” means the operation is fully determined by the algorithm alone once it is known — which, per Kerckhoffs’s principle, must be assumed.

Primitive	Key dependence	Effective states
ophex	none (encoding only)	1 (zero)
opbase64	none (encoding only)	1 (zero)
opatbashtr	none	1 (zero)
opunicodeshift	none (delta hardcoded ± 5)	1 (zero)
oprailfence	none (rails hardcoded per CL)	1 (zero)
opblockrotate	none (n hardcoded to 3)	1 (zero)
opsplitreverse	none	1 (zero)
opfullreverse	none	1 (zero)
opcolumnar	digit-sum mod 5	5 (columns $\in \{4..8\}$)
opxorkey	first digit only	10
oprollingxor	first digit only (seed)	10
opsbox	weighted digit-sum ($\sum d_i \cdot (i+1)$)	325
opblockxorcbc	IV = literal KOD8KEY string	fixed, non-random
opblockshuffle	digits mod block count, deduplicated	$\leq 8!$, usually far smaller
opvigenetr	full key, period-8 repeating shift	10^8 (nominal)
opbase36	full key, period-8 repeating shift	10^8 (nominal)
opkeystreamxor	full key as LCG seed	10^8 (nominal)

(a) Six of seventeen primitives are not cryptographic at all. opatbashtr, opunicodeshift, oprailfence, op_blockrotate, opsplreverse, and opfullreverse never inspect

```

op_atbash_tr    -> references KOD8_KEY: False op_unicode_shift -> references
KOD8_KEY: False op_rail_fence    -> references KOD8_KEY: False op_block_rotate
    -> references KOD8_KEY: False op_split_reverse -> references KOD8_KEY: False
op_full_reverse -> references KOD8_KEY: False

```

KOD8KEY. We confirmed this by static inspection of every function body:

1
2
3
4
5
6

In CL1 (plain text), for example, four of the eight advertised layers — atbashtr, blockrotate, unshift, and fullreverse — fall into this category. Under Kerckhoffs’s assumption (the attacker knows the algorithm, only the key is secret), these steps contribute *no* confidentiality whatsoever: they are fixed, publicly known, and instantly invertible by anyone who has read this report or the source. Their function is purely to increase visual and structural obfuscation, not adversarial work factor.

(b) The columnar-transposition parameter has only five possible values. opcolumnar derives its column count from $(\text{sum}(\text{int}(d) \text{ for } d \text{ in KOD8KEY}) \% 5) + 4$. Since this is bounded between 4 and 8

regardless of the underlying key, we confirmed empirically that 100,000 randomly sampled eight-digit keys produce only the five values $\{4,5,6,7,8\}$ — meaning an attacker can exhaustively enumerate every possible columnar transposition used by CL4 in five trials.

(c) The S-box seed collapses an 10^8 key into 325 states. The deterministic Fisher–Yates shuffle that

```
seed = sum(int(c) * (i + 1) for i, c in enumerate(KOD8_KEY))
```

produces the substitution box used by CL2, CL5, and CL6 is seeded with a positionally weighted digit sum,

1

For an eight-digit key this expression is bounded between 0 (all-zero key) and $9 \times (1+2+\dots+8) = 324$, i.e. 325 possible seed values, and therefore at most 325 distinct S-boxes, independent of which of the 10^8 nominal keys is in use. We validated this by sampling 200,000 random eight-digit keys and recording the

```
Distinct S-box seeds observed in 200,000 random keys: 287
Theoretical maximum distinct seeds: 325
```

resulting seed:

1

2

This is close to the theoretical ceiling, as expected once the sample size exceeds the state space by two orders of magnitude. Practically, this means an attacker facing any CipherList that uses opsbox never needs to search 10^8 keys for that layer — a 325-entry precomputed table of candidate S-boxes suffices, and each candidate can be tested in microseconds.

5 Fixed Initialization Vector in the CBC-Style Layer

opblockxorcbc is explicitly modeled on CBC-mode block chaining, with each plaintext block XORed against

```
iv = (KOD8_KEY * ((block_size // len(KOD8_KEY)) + 1))[:block_size]
```

the *previous ciphertext block*, seeded by an initialization vector:

1

With the default blocksize=8, which is also the length of KOD8KEY, this expression reduces to `iv ==`

```
block_xor_cbc IV == KOD8_KEY literally: True '86247931'
```

KOD8KEY verbatim — we confirmed this by direct execution:

1

This has two consequences that a real block cipher’s CBC mode is specifically designed to avoid:

1. The IV is not random and is never varied per message. Every invocation of every CipherList that uses this primitive (CL2, CL3) uses the identical IV. Combined with the determinism shown in Section 6, this guarantees that identical inputs always chain identically from the very first block.
2. The IV is not distinct from the key. In a textbook CBC construction the IV need not be secret, but reusing key material as the IV needlessly collapses two independent security parameters (confidentiality key,

chaining diversifier) into one, removing any benefit CBC-style chaining would otherwise provide against identical-prefix leakage across messages encrypted under different (hypothetical, rotated) keys.

6 Determinism and the Absence of Semantic Security

Because every layer of every CipherList is either unkeyed (Section 4) or keyed only by the static KOD8KEY with no per-message nonce, salt, or random IV anywhere in the chain, KOD8 is fully deterministic: identical

Same plaintext -> same ciphertext (deterministic)? True

plaintext always produces byte-identical ciphertext.

1

This was confirmed by encrypting the string "THE MEETING IS AT NOON" twice, via two independent Kod8 instances, and observing identical output both times. Deterministic encryption fails the standard notion of semantic security (IND-CPA): an observer who cannot decrypt a single message can still detect message repetition, build a frequency profile of recurring ciphertexts across a captured communication stream, and mount straightforward replay or dictionary-matching attacks — all without ever recovering the key.

7 Absence of Integrity and Authentication

No primitive in KOD8 produces or checks a message authentication code, and decrypt() performs no

```
def op_hex(text: str, encrypt: bool, **_) -> str:
    if encrypt:
        return text.encode("utf-8").hex().upper()
    try:
        return bytes.fromhex(text).decode("utf-8")
    except Exception: return text
```

integrity validation on its output. Worse, several primitives are written to fail *silently*. For example:

1

2

3

4

5

6

7

A malformed or tampered ciphertext does not raise an error here; it silently falls through and returns the (still-encrypted, or partially-mangled) input unchanged, which then continues through the remaining seven reverse steps and is ultimately displayed to the user as if it were valid plaintext. opbase64 exhibits the identical pattern. This is a textbook malleability weakness: an attacker with write access to the ciphertext channel can flip, truncate, or splice bytes — particularly within the XOR-based layers, which are linear and therefore preserve attacker-controlled deltas directly into the recovered plaintext — with no cryptographic mechanism to detect the tampering.

8 Cleartext CipherList Disclosure

KOD8’s auto-detection mechanism (autodetect) is presented as adding an extra layer of ambiguity: a ciphertext could, in principle, have been produced by any of six different eight-step chains. In the reference implementation this ambiguity does not survive contact with the persistence layer. `kod8ui.py` writes every

```
f.write(f"# KOD8          cipher_id: {cid}\n{enc}")
```

saved file as:

1

```
if line.startswith("# KOD8 cipher_id:"):
    cid = line.split(":")[1].strip()
```

and the decrypt path reads this header back out directly:

1

2

Every .kod8 file therefore discloses, in cleartext, exactly which of the six eight-step chains was used. This reduces the attacker’s problem from “recover a key against an unknown one-of-six chain” to “recover a key against one specific, fully known chain” — eliminating what would otherwise be the system’s only source of genuine combinatorial ambiguity beyond the key itself.

9 Empirical Key Recovery

To establish a concrete, conservative upper bound on attack cost, we implemented a blind known-plaintext brute-force attack against CL1 (the plain-text CipherList) that makes *no* use of the entropy-collapse findings in Section 4 — i.e. it searches the full nominal 10^8 keyspace as if every digit of KOD8KEY contributed independently to every step, which Table 1 shows is not actually the case.

9.1 Method

A plaintext ("ATTACK AT DAWN TOMORROW") was encrypted once under the reference key to obtain a known plaintext/ciphertext pair. For each of N randomly sampled eight-digit candidate keys, we re-derived the dependent S-box exactly as the engine does at import time, ran the full CL1 decryption chain, and compared the result to the known plaintext. Per-trial wall-clock time was measured directly and extrapolated linearly to the full 10^8 -key search.

9.2 Results

```
2000 trials in 0.1676s -> 83.80 microseconds/trial
Found at trial 1000 (candidate = true key '86247931')
Estimated time for exhaustive 10^8 search
(single core, unoptimized reference Python): 8379.7s ~ 2.33 hours
```

1

2

3

4

This is a *worst-case, no-shortcuts* figure using an interpreted, unoptimized reference implementation on a single CPU core. It is an upper bound in every practical sense:

- **Parallelization.** The search is embarrassingly parallel (each candidate key is independent). Across 32 cores, 2.33 hours falls to under 4.4 minutes; on commodity cloud infrastructure or a GPU-hosted implementation, recovery drops to seconds.
- **Algorithmic shortcuts.** Section 4 shows that `opxorkey` and `oprollingxor` depend only on the first key digit (10 candidates), and that the S-box depends on only 325 candidate seeds rather than 10^8 keys. A cryptanalyst exploiting these facts — for example, solving `opxorkey` directly via a single-byte-XOR frequency or known-plaintext attack, and enumerating the 325-entry S-box table rather than the full key — reduces several CipherLists’ effective search space by five to six orders of magnitude, to the point of near-instant recovery on a laptop.
- **No implementation optimization was attempted.** The benchmark uses pure-Python string operations with no vectorization, caching, or compiled extensions; a systems-level reimplementation (C, Rust, or even NumPy-vectorized Python) would be expected to improve per-trial latency by one to two further orders of magnitude.

10 Per-CipherList Summary

Table 2 aggregates the findings above per CipherList, identifying the weakest key-dependent primitive in each chain — i.e. the layer a cryptanalyst would target first.

Table 2: Weakest link per CipherList and the resulting practical attack surface.

ID	Input class	Weakest keyed primitive	Practical implication
CL1	Plain text	opxorkey (10 states)	Trivial single-byte-XOR solve; 4 of 8 steps unkeyed
CL2	Image	opsbox (325 states) + fixed CBC IV	S-box table lookup; deterministic chaining from block 0
CL3	Video	fixed CBC IV (opblockxorcbc)	Identical file prefixes leak identical ciphertext prefixes
CL4	Document	opcolumnar (5 states)	Column count enumerable in 5 trials
CL5	Numeric/data	opsbox (325 states)	S-box table lookup dominates over full-key search
CL6	Experimental (fallback)	opsbox (325 states)	Same as CL2/CL5

11 Discussion

Taken together, these findings support a specific and fairly narrow conclusion: KOD8 is a wellconstructed *obfuscation* layer and a legitimate example of classical product-cipher design, but it does not meet the security expectations implied by its “8-layer, 6-strategy” framing, and it should not be relied upon to protect data against a motivated adversary.

This is not a criticism unique to KOD8; it is the expected outcome for *any* cipher built from XOR, substitution, and transposition primitives over a small numeric key without a formal security proof, a vetted

key-derivation function, randomized nonces, and an authenticated-encryption construction. Two observations frame this constructively rather than dismissively:

1. The stated design goal — resisting casual, automated, pattern-based decoding (e.g. by an LLM skimming ciphertext for recognizable structure) — is a materially different, and much weaker, goal than confidentiality against a targeted cryptanalytic adversary. Under the former goal, several of the “zero-entropy” primitives in Table 1 (Atbash, transpositions, codepoint shifting) are entirely reasonable: they destroy exactly the kind of surface-level regularity that pattern-matching or naive statistical decoding would key off, even though they add nothing against a human cryptanalyst who reads the source. Evaluated against *that* narrower goal, KOD8 likely performs its intended job.
2. The gap between the two goals is exactly where the current implementation is most exposed, principally via T1 (hardcoded key) and the cleartext CipherList header — both of which are implementation choices, not inherent to the cipher-chain design, and both are straightforward to remediate without redesigning the eight primitive families.

12 Recommendations

1. Never treat KOD8 in its current form as confidentiality protection for sensitive data. If genuine confidentiality is required, use a vetted AEAD construction (AES-256-GCM or ChaCha20-Poly1305) with a key derived via a proper KDF (Argon2id, scrypt, or PBKDF2 with a modern iteration count) from a user-supplied passphrase — not a hardcoded constant.
2. If KOD8 is retained for its stated obfuscation purpose, document that scope explicitly in the code and UI so future users do not mistake it for a security control, and stop persisting the CipherList identifier in cleartext alongside the ciphertext — derive it instead from a value only the key holder can compute (e.g. include it inside the encrypted payload).
3. Introduce a random, per-message nonce or IV into any retained CBC-style or keystream layer, transmitted alongside (not derived from) the ciphertext, to eliminate the determinism identified in Section 6.
4. Add a MAC or AEAD tag (even a simple HMAC-SHA256 over the final ciphertext) so that tampering is detected rather than silently decrypted into plausible-looking garbage, closing the failure mode in Section 5.
5. Re-derive the S-box seed from the full key material (e.g. via a cryptographic hash of KOD8KEY rather than a positional digit sum) to eliminate the 325-state collapse in Section 4.

13 Conclusion

We performed a source-grounded, empirically validated cryptanalysis of KOD8. Static analysis of all seventeen primitives shows that only three of the eight steps in a typical CipherList meaningfully depend on the key, and that the two primitives most resembling “real” cryptographic components — the S-box and the CBC-style block XOR — each contain implementation-level defects (a 325-state seed collapse and a literal-key-as-IV reuse, respectively) that remove most of the protection their design intends to provide. A conservative, no-shortcuts, known-plaintext brute-force attack recovers the full eight-digit key against the plain-text CipherList in under 2.4 hours on a single CPU core with an unoptimized reference implementation, and the structural weaknesses identified here reduce that further to seconds. Combined

with a hardcoded global key and a cleartext CipherList header in the reference file format, KOD8 is best classified as a classical obfuscation cipher suitable for resisting casual or automated pattern-based inspection, not as a cryptographically secure scheme, and it should be labeled and used accordingly.